BIT502 Fundamentals of Programming Assessment 1

Weighting

20%

Learning outcomes

1 Create simple applications using fundamental programming logic, constructs and mathematical operations to solve business problems.

Instructions

Complete and submit your assessment according to the Open Polytechnic's Assessments webpage. This includes information on academic integrity, formatting, word limits and referencing.

- Include your name, student number and the assessment number.
- Number your pages.

Submission

- Submit your assessment in one zip file.
- Submit your work through your iQualify course.
- Emailed assessments will not be accepted.
- You will receive an automated notice advising you of your successful submission.

By submitting your assessment, you confirm that it is your own, original work.

© The Open Polytechnic of New Zealand

Introduction

The main objective of this assessment is to apply the programming fundamentals skills you have developed to create a console application using Python. Your console application must address the needs of the business in the case study.

To create and access your working project, follow these steps.

1. Create a new directory.
2. Name your directory "BIT502 A1 - [your name]".
3. Create one Python code file inside the directory, named "BIT502_AS1_[your first name_your last name].py"
4. Use a code editor to open the directory. Preferably Visual Studio Code.
5. Edit your Python file.
6. Save your Python file.

Note: We recommend using Visual Studio Code instead of Jupyter Notebook as the code editor to complete this assessment.

Before you submit

- The code file should be submitted as a zip file.
    - Include your course code (BIT502), name, student number, and assessment number as a comment, at the top of your code.
- Ensure your zip file includes:
    - your Python file.

After zipping your file, reopen it to make sure it works and contains the correct file.

Read the following case study to complete the tasks. Refer to the Marking Schedule for specific information about marking.

© The Open Polytechnic of New Zealand

Case study

A local bookstore, The Aurora Archive, has been in business for several months and is seeing a steady increase in members. In addition to selling books, they have a few services that require the use of a membership subscription; these include:

- access to a private reading area
- a kid friendly environment that encourages reading becoming a lifelong habit
- various book and reading related activities.

The owners have asked you to develop some programs to help improve their administration service to save them time and reduce errors.

Digitising membership plans and costs

Currently, the administration team uses paper-based documentation, such as forms and files. They would like to gradually move to a digital format, starting with automating how membership plans and costs are calculated.

The current system requires staff to manually calculate costs for new members. This takes time and can occasionally lead to mistakes, which are difficult to correct once the new member has signed up.

The owners see this as an opportunity to implement a software solution to help staff calculate the cost of membership for new members.

While the bookstore has a computer capable of running console applications, no database has been set up yet. This program would allow staff and clients to check the price for the various plans and options available for membership. However, since there is no database available, the program will only be informational and perform a few simple functions.

Kids' Reading Challenge

To promote the fun of reading, the store challenges their young members to read a book the store selects periodically. The kids compete to see who has read the most pages in that period. For this program, the user will enter the number of pages they read each day for the five weekdays. The program then provides them with a reader rank based on the total pages.

The store owners want this to be a fun and encouraging opportunity for the young clients as they progress on their reading journey.

© The Open Polytechnic of New Zealand

Task 1: Creating the main menu

Create a main console menu that displays when the user runs the application. The menu should display the following four options to the user, along with an appropriate menu header.

1. Membership Plans
2. Optional Extras
3. Reading Challenge
4. Aurora-Picks Rental Calculator
5. Exit the program

Make sure that you place a number next to each option in the displayed menu. Ask the user to pick an option, then run the code for the selected option. The option should correspond with the number displayed. Any sub-menus will also use the same method.

Ideally, using the menu should be convenient for the user.

Note: When moving from one menu to another, make sure to clear the console. This will prevent the console from becoming cluttered with previous menus and sub-menus.

Error handling: If a user selects an option other than 1 to 5, display an appropriate error message and then present the menu again for selection.

(Task 1 total: 15 marks)

© The Open Polytechnic of New Zealand

Task 2: Creating the membership plans sub-menu

After the 'Membership plans' option is selected from the main menu, the program will display the price of the membership plans available. Users should be able to pick an option from the menu and view the membership cost for that option.

Part A: Selecting the membership plan

Create a sub-menu with an appropriate header that displays to the user and shows the following options:

1. Standard
2. Premium
3. Kids
4. Return to main menu
5. Exit

The program should wait for the user to pick an option, then move on to displaying the result of the option selected, returning to the previous menu if option 4 is selected, or exiting the application if option 5 is selected. Ensure the menu is convenient for the user.

(10 marks)

Part B: Displaying the membership cost

There are three membership types (standard, premium, and kids). The monthly base costs for each are:

1. Standard: $10
2. Premium: $15
3. Kids: $5

After the membership type option is selected, the program should display the monthly cost for the option selected along with a brief description of the plan in your own words. The standard plan provides basic membership, while the premium plan provides access to book discounts and special sales, the kids' plan is the same as standard but only if they are 12 or younger.

Also display the cost of annual membership; annual membership receives a discount of one month free, making the cost only 11 months instead of 12.

As these prices are subject to change in the future, use good programming practices to make editing the values easy.

The process should be convenient for the user.

© The Open Polytechnic of New Zealand

Note: After displaying the details for any membership type, include a prompt such as “Press any key to continue” or a similar appropriate message. Once a key is pressed, the membership plan selection should be displayed again after clearing the previous display.

Error handling: If a user selects an option other than 1 to 5, display an appropriate error message and then present the menu again for selection.

Part C: Returning to the previous menu

When the user selects this option, the application returns to the original menu.

Part D: Exiting

Closes the application.

(15 marks)

(Task 2 total: 25 marks)

© The Open Polytechnic of New Zealand

Task 3: Optional extras

After 'Optional extras' is selected from the main menu, the program will display a list of products and services. The user will be required to select each optional extra that they would like to add to their plan, before seeing the total cost for the optional extras.

The optional extra prices are a monthly cost and separate from the base membership cost.

Part A: Displaying information about the optional extras

The program should display the following prices in an appropriate format to the user. Add any additional messages or information you feel will improve the user experience.

- Book rental is $5
    - They can borrow one book at a time from a select range of older books. Returning the book will allow them to borrow another up to twice per month. This rental option is separate from the Aurora-Picks rental system.
- Private area access is $15
    - The member will have access to the second floor of the building where a quiet reading area has been provided with comfortable seating.
- Monthly booklet is $2
    - At the start of the month, the shop sends out a booklet to the member covering news, events, reviews, and upcoming releases for the month.
- Online ebook rental is $5
    - Members can use an e-reader to access a selection of books and borrow these. Only one member can borrow each book at a time, books are automatically returned after 7 days.

Part B: Prompting for the user options

Once the program has displayed the information, it should prompt the user with a yes or no question for each of the extras. You will need to ask once for each optional extra, for example, "Would you like the monthly booklet for $2 (yes/no)?".

Note: When prompting the user for input, you can accept "yes" or "y", and "no" or "n" to improve user convenience. Case should be ignored.

Part C: Displaying the calculated total

After the user has selected 'yes' or 'no' for each of the four options, the program should show them the options they selected and the total cost for the extras in a clear format. Once the calculations are completed, the program should prompt the user to return to the main menu when they press enter.

© The Open Polytechnic of New Zealand

Good programming practices should be followed, and the process should be convenient for the user.

Note: Please refer to the figure below for an example display of the available extras and selection options. Select an appropriate format for both the display and message content, keeping in mind that this is only a sample. Also, ensure that proper error handling is implemented to address cases where an invalid selection is made (refer to the figure for a sample error handling).

```
Welcome to extras.....Add more appropriate message
Press any key to proceed with the selection..
---------------------------------
Do you want Extra 1 (or some appropriate prompt)?: t
Wrong choice. Please press any key to re-enter your choice..
Do you want Extra 1 (or some appropriate prompt)?: no
---------------------------------
Do you want Extra 2 (or some appropriate prompt)?: ye
Wrong choice. Please press any key to re-enter your choice..
Do you want Extra 2 (or some appropriate prompt)?: YES
---------------------------------
Do you want Extra 3 (or some appropriate prompt)?: No
---------------------------------
Do you want Extra 4 (or some appropriate prompt)?: y
---------------------------------
Thank you for the selection. The total cost of the selected extras is $ 20
Press any key to return to the main menu.
```

(Task 3 total: 20 marks)

Task 4: Kids' Reading Challenge

After 'Kids' Reading Challenge' is selected from the main menu, the program will display a convenient way for the user to enter their reading progress for the challenge.

To encourage regular reading, the challenge will have kids record the number of pages they read each weekday, Monday through Friday. They will need to enter a number for each of these five days.

After entering all five days, the total pages are calculated, and the user will be given a ranking based on the total pages.

Provide a good user experience, making the program convenient and also make sure to account for potential problems while maintaining good practices.

Sample output

Welcome to the Reading Challenge. Please enter your time for each day.

Hit enter to start…

Monday: 20

Tuesday: 15

Wednesday: 13

Thursday: 10

Friday: 19

Total pages read: 77

Average pages per day: 15.4

You ranked Bronze!

Monday was your biggest reading day!

To reach the next rank you need to read 23 more pages. Good luck!

Press any key to return to the main menu…

Part A: Gathering pages read

The program should display an appropriate message to welcome the user to this section. The user is asked to input the pages they have read for each of the five days, Monday through Friday. Make the process convenient and user friendly where possible.

Note: Please ensure proper error checking similar to that performed on the extras inputs. Both integer and decimal values for the number of pages read are valid, so entries like 23.7

© The Open Polytechnic of New Zealand

pages are acceptable. However, any input that is not a valid number, including empty values, should be considered invalid.

Part B: Calculating the results

The program should tally the total pages read by the user and provide some feedback based on their results. Although it should be a simple addition, make sure to keep the values separate so you can find the other information.

After the calculation, the user is ranked in the following four categories:

- Bronze: Less than or equal to 25 pages
- Silver: More than 25 pages and less than or equal to 50 pages.
- Gold: More than 50 pages and less than or equal to 100 pages.
- Platinum: More than 100 pages.

Part C: Displaying results

The program should display the following to the user:

- Total pages read in a week.
- Average pages read per day.
- The day(s) with the most pages read, showing all relevant days if there are multiple. So if the highest pages read is 23 pages on Tuesday and Sunday, then show Tuesday and Sunday.
- Reading rank in an appropriate format.
- Number of pages needed to reach the next rank. If at the highest rank, display a message indicating that there is no further rank, and they are at the highest rank.
- ***Additional display only if applicable:*** The current record is 150 pages read in a week, if the user breaks this record they should receive a special congratulatory message.

Part D: Returning to the previous menu

When the user has finished, the application returns to the main menu.

(Task 4 total: 20 marks)

© The Open Polytechnic of New Zealand

Task 5: Aurora-Picks Rental Calculator

The store offers a range of specially selected books periodically that customers may rent; these are hand-picked by the staff.

After 'Aurora-Picks Rental Calculator' is selected from the main menu, display the relevant information to the user and allow them to select from a sub-menu with two options:

1. Enter rental period
2. Return to main menu

When selecting the first option, ask the user for the number of days they wish to rent a book for; calculate and display the total cost. Return them to the sub-menu when they are ready.

Selecting the second option will return them to the main menu.

The rental price is as follows:

- The base cost is $1 per day for the first 3 days
- After 3 days the cost per day is reduced to $0.80
- After 8 days the cost per day is reduced to $0.50

The price accumulates each day. For example, if a book is rented for five days; the total cost is $3 for the first three days, and $1.60 for the next two days, for a total of $4.60.

Example: 3 + 0.8 + 0.8 = 4.6

There are a few rules with rental:

- Minimum rental period is 3 days
- Maximum rental period is 21 days
- If renting for 21 days, the cost is fixed to $12

Display any necessary warnings or errors as necessary.

An error will occur if the input is not an integer (e.g., entering 5.7 days would be invalid). Additionally, a warning message will be displayed if the input is less than 3 days or greater than 21 days. In both cases, the user should be prompted to re-enter a valid number of days.

The costs may change in the future, ideally it will be easy for the client to update these values.

Provide a good user experience, making the program convenient and also make sure to account for potential problems while maintaining good practices.

(Task 5 total: 20 marks)

© The Open Polytechnic of New Zealand

Quality control

Essential functions of your application

Ensure that your program has followed good programming practices and performs the required functionality.

Your Python console application must include the following.

- The user should be able to navigate between the main menu and carry out other operations without needing to terminate the application (unless they select the option to exit).
- Menus should have a heading to make it clear which section of the application the user is in.
- The menu should prompt the user for input. For example: "Enter your selection".
- Selecting an option that is not on the menu should warn the user and allow them to select another option from the same menu.
- A message should be displayed before the user exits the program.

Appropriate formatting that conveys meaningful output should be used. Use punctuation, suitable text alignment and symbols to display all necessary information to the user. For example, use a dollar sign before an output that is a cost.

Good programming practices

Your code should follow good programming practices, including:

- organising the code in a clear manner
- using meaningful variable naming
- minimising duplicate code
- using your own methods where sensible
- performing validation where necessary
- handling unexpected input or user mistakes appropriately
- ensuring the program does not crash or produce errors
- avoiding hard-coding values where possible
- adding comments to explain the complex sections of the code.

Although well written code should not require much if any commenting, as the programs may undergo future development you have been asked to provide some comments to help explain any code that may not be obvious, such as calculations and formula in the code. You should not comment on every line.

Ensure your program is free from errors and compiles correctly.

© The Open Polytechnic of New Zealand

Marking schedule

Task 1	12 – 15 marks	10 – 11.5 marks	7.5 – 9.5 marks	6 – 7 marks	1 – 5.5 marks
Creating the main menu	The main menu is fully functional with validation and error checking. The user experience is excellent and good programming practices have been followed throughout.	The main menu functions as expected. Validation and basic error checking is performed. The user experience has been well considered. One or two errors exist or some good practices may not have been followed.	The main menu has been developed and allows the user to select all options.	The main menu is being displayed to the user and allows the user to select at least one option or request user input.	The main menu is being displayed to the user but is missing a lot of functionality.
Task 2: Part A	**8 – 10 marks**	**6.5 – 7.5 marks**	**5 – 6 marks**	**4 – 4.5 marks**	**1 – 3.5 marks**
Selecting the membership plan	The sub-main menu is fully functional with validation and error checking. The user experience is excellent and good programming practices have been followed throughout.	The sub-menu functions as expected. Validation and basic error checking are performed. The user experience has been well considered. One or two errors exist or some good practices may not have been followed.	Sub-menu performs the minimum functionality allowing the user to select all options.	View membership cost option is partially functional. Option allows for user input. Significant errors in code and functionality exist.	View membership cost option has been started but is missing a lot of functionality.
Task 2: Part B, C, D	**12 – 15 marks**	**10 – 11.5 marks**	**7.5 – 9.5 marks**	**6 – 7 marks**	**1 – 5.5 marks**
Displaying membership cost, returning to previous menu orexiting	All options function as expected The user experience is excellent and good programming practices have been followed throughout.	All options function as expected. The user experience has been well considered. The user experience has been well considered. One or two errors exist or some good practices may not have been followed.	All options are selectable and perform the related function. Various improvements could be made to the user experience, errors exist, or good practices may not have been followed.	Some of the functionality is working but largely incomplete or has serious issues and errors.	Some attempts have been made with the functionality, but a lot is missing or incomplete.

© The Open Polytechnic of New Zealand

Task 3	16 – 20 marks	13 – 15.5 marks	10 – 12.5 marks	8 – 9.5 marks	1 – 7.5 marks
Optional extras	The user can decide for each option and calculation is performed as expected. The user experience is excellent and good programming practices have been followed throughout.	The user can decide for each option and calculation is performed as expected. The user experience has been well considered. One or two errors exist or some good practices may not have been followed.	The user can decide for each option and calculation is performed as expected. Improvements could be made to the user experience, errors exist, or good practices may not have been followed.	Information about the extras is displayed and wellformatted. The user can return to the main menu. Errors exist in functionality or calculation.	Information about the extrasis displayed. Functionality is still incomplete, but some parts are attempted.
Task 4	**16 – 20 marks**	**13 – 15.5 marks**	**10 – 12.5 marks**	**8 – 9.5 marks**	**1 – 7.5 marks**
Kids' Reading Challenge	The challenge and additional features are functioning and complete. The user experience is excellent and good programming practices have been followed throughout.	The challenge and additional features are functioning and complete. The user experience has been well considered. One or two errors exist or some good practices may not have been followed.	The basic challenge can be completed with results displayed. Most additional functionality, such as the record or rankings is functioning. Improvements could be made to the user experience, errors exist, or good practices may not have been followed.	The minimum challenge functionality is partially working, the user can enter the 5 days and totals are calculated and displayed. Additional functionality is missing, such as breaking the record number or rankings.	Some attempts have been made with the functionality, but a lot is missing or incomplete.
Quality control	**16 – 20 marks**	**13 – 15.5 marks**	**10 – 12.5 marks**	**8 – 9.5 marks**	**1 – 7.5 marks**
Book Rental Calculator	Rental calculation is fully functional. The user experience is excellent and good programming practices have been followed throughout.	Rental calculation is fully functional. The user experience has been well considered. One or two errors exist or some good practices may not have been followed.	Rental calculation functionality is working, the user can enter a value and receive calculated output. Various improvements could be made to the user experience, errors exist, or good practices	Some of the functionality is working but largely incomplete or has serious issues and errors.	Some attempts have been made with the functionality, but a lot is missing or incomplete. Menu is displayed and some attempt with displaying rental period is made.

© The Open Polytechnic of New Zealand

			may not have been followed.		

© The Open Polytechnic of New Zealand